

Adjust Java SDK 集成开发指南

文档 v1.0.1 | SDK 1.0.0

文档说明

本指南面向站网平差系统的集成人员，说明 adjust-java-lib 的安装、Java 公共接口、ENU 与 WGS84 模型、JSON 协议、质量诊断和运行注意事项。输入为控制坐标与基线向量，输出为平差坐标、精度、基线残差和告警。

文档依据 Git bf85d95 的源码、构建产物及 2026-09-10 本机验证编写。Java SDK 版本以 pom.xml 为准，为 1.0.0；本机原生 C ABI 版本为 1.0.0。

使用范围

核心接口适用于有固定控制点的 ENU 或 WGS84 站网。随机控制先验、自由网、稀疏求解和完整协方差等高级能力使用原始 JSON 接口，并遵循对应原生库协议。坐标标准差和合成算例的一致性不能替代实测站网的精度验收。

本轮使用确定性合成数据，在 macOS arm64 上通过完整构建、3 项现有测试和 34 组补充验证。加权平差、闭环与 WGS84 的整体检验通过，默认 Huber 粗差样本收敛且降低偏差；数据构造、数值容差和完整记录见配套测试报告。

文档修改记录

版本	日期	说明
v1.0.0	2026-09-10	依据实际接口与验证结果编写；沿用 DMonitor 文档版式。
v1.0.1	2026-09-10	更新合成测试数据、收敛与整体检验结果；精简集成说明。

一、SDK 简介

Adjust Java SDK 通过 JNA 调用 libAdjust 的同步 C 接口，使用 UTF-8 JSON 传输请求与响应。Java 层负责对象转换、传输边界检查和结果解析；数值求解及站网业务校验由原生库负责。

层级	入口	主要职责
业务服务	AdjustLibService	普通 Java 对象或 Spring Bean；提供强类型与原始 JSON 两类入口。
协议工具	JsonUtils	构造核心请求、校验字符串、解析 ok/result/error 信封。
原生绑定	AdjustLibrary	加载 Adjust 动态库；映射 getVersion、startAdjust、startAdjustWgs84。

接口选择

固定控制点和常规基线站网推荐使用强类型接口。需要保留完整矩阵、坐标先验、分组和高级求解配置时，使用字符串接口自行处理完整结果。两类入口均同步返回，不提供回调注册、任务句柄或流式订阅。

二、名词解释

名词	含义
ENU	东、北、天坐标或向量；核心 ENU 入口中全部观测必须使用同一套坐标轴。
WGS84 / ECEF	WGS84 经纬度和椭球高；ECEF 为地心地固 XYZ 直角坐标。
控制点	精确固定坐标，不作为待估参数；不要用固定点代替有不确定度的随机先验。
基线向量	终点减起点；单位米，方向由 from → to 决定。
协方差	3×3 对称正定矩阵，单位平方米，包含分量方差及相关性。
自由度 / sigma0	冗余观测分量数；sigma0 为后验单位权标准差。
鲁棒平差	降低异常基线的权值；仍需检查迭代收敛与质量信息。

三、快速开始

1. 运行与编译环境

项目	要求
Java	编译 release=20；运行需 JDK/JRE 20 或以上，本轮 JDK 21.0.9。
Maven	工程带 Wrapper 3.9.16，可使用 ./mvnw；建议统一通过 Wrapper 构建。
依赖	普通 Java 运行依赖 JNA 5.12.1、Gson 2.10.1；Spring Boot 3.2.4 支持为可选依赖。
输入	固定控制点、唯一站点 ID、基线端点、方向向量及有效协方差。
原生库	与操作系统、CPU 架构相匹配的 libAdjust；当前 API 无许可证参数。

2. 原生库加载

本轮使用内置 darwin-aarch64/libAdjust.dylib，在 macOS arm64 进程中实测。JNA 可从 classpath 资源加载该库；外部库目录配置见后文。

3. 构建与安装

```
./mvnw clean verify
./mvnw install
```

```
<dependency>
  <groupId>com.navfirst</groupId>
  <artifactId>adjust-java-lib</artifactId>
  <version>1.0.0</version>
</dependency>
```

产物为普通依赖 JAR：target/adjust-java-lib-1.0.0.jar。已验证可直接从 classpath 导入；不需要提取 BOOT-INF。依赖坐标应与实际构建的 1.0.0 保持一致。

三、快速开始（续）

4. 创建服务并调用

```
AdjustLibService sdk = new AdjustLibServiceImpl();
System.out.println(sdk.getVersion());
ENUMNetworkResult result = sdk.startAdjust(problem);
// See Appendix A for the complete problem.
```

附录 A 的双观测算例固定 $A=(0,0,0)$ ，对 $A \rightarrow B$ 输入 $(10,2,0.5)$ 和 $(12,4,1.5)$ ，各分量标准差分别为 1 m、2 m。非等权平差得到 $B=(10.4,2.4,0.7)$ m，可作为首次集成的可核对答案。

5. Spring Boot 3 自动配置

```
import com.navfirst.adjust.lib.services.AdjustLibService;
import org.springframework.stereotype.Service;

@Service
public class NetworkAdjustmentService {
    private final AdjustLibService sdk;
    public NetworkAdjustmentService(AdjustLibService sdk) {
        this.sdk = sdk;
    }
    public String nativeVersion() { return sdk.getVersion(); }
}
```

AdjustAutoConfiguration 通过 AutoConfiguration.imports 注册。缺省开启；已有 AdjustLibService Bean 时不再创建默认服务。默认服务构造时触发动态库加载。

```
# application.yml
adjust:
  enabled: false
```

该配置关闭自动配置注册。AdjustLibServiceImpl 本身也有 @Service；若应用额外扫描 SDK 实现包，组件扫描仍可能注册它。建议使用默认自动配置，无需扩大扫描范围。

6. 外部动态库

```
java -Djna.library.path=/opt/adjust/lib \
    -Djna.encoding=UTF-8 -jar your-application.jar
```

JNA 默认通过 Native.load("Adjust") 查找并加载资源。外部库目录应包含当前平台正确命名的 libAdjust 文件。UTF-8 是接口协议，建议显式设置 jna.encoding=UTF-8。

四、Java 公共接口

1. AdjustLibService

方法	返回与行为
getVersion()	String；返回当前实际加载动态库的 C ABI 版本，与 Java SDK 版本分开管理。
startAdjust(String requestJson)	String；校验传输边界后原样调用，保留完整响应信封。
startAdjustWgs84(String requestJson)	String；WGS84 JSON 入口，支持 geodetic_problem 信封或直接问题对象。
startAdjust(ENUNetworkProblem problem)	ENUNetworkResult；默认核心配置，无超时、关闭鲁棒。
startAdjust(ENUNetworkProblem problem, AdjustOptions options)	ENUNetworkResult；options 可为 null，原生失败转为 AdjustException。
startAdjustWgs84(GeodeticNetworkProblem problem)	GeodeticNetworkResult；默认核心配置。
startAdjustWgs84(GeodeticNetworkProblem problem, AdjustOptions options)	GeodeticNetworkResult；每条基线显式设置 frameWGS84。

这 7 个 Java 方法都是同步调用。字符串入口不执行强类型默认配置注入，也不会自动把 ok=false 转成业务异常。传输校验失败和动态库加载错误仍会直接抛出。

2. 调用前后的责任

调用方负责坐标基准、单位、基线方向、观测协方差和站点连接关系。调用完成后，先检查诊断与告警，再读取坐标或精度；成功返回对象只说明求解流程完成，不保证质量门槛已达到。

```
try {
    var options = AdjustOptions.builder().timeoutMs(5000L).build();
    var result = sdk.startAdjust(problem, options);
    // Check convergence, warnings and global test; see Chapter 6.
} catch (com.navfirst.adjust.lib.exceptions.AdjustException e) {
    System.err.printf("code=%s field=%s id=%s message=%s\n",
        e.getCode(), e.getField(), e.getId(), e.getMessage());
}
```

五、输入输出模型

1. ENUNetworkProblem

顶层字段 stations 为 List<Station>，baselines 为 List<Baseline>。固定站与自由站共用同一个 Station 模型，所有坐标和向量使用同一套公共 ENU 坐标轴。

Java 字段	JSON 字段 / 类型	约束与说明
Station.id	id / String	唯一站点 ID；基线端点必须引用已存在站点。
Station.fixed	fixed / boolean	true 表示精确固定；默认 false。
Station.knownENU	known_enu / Geometry.ENU	固定站必填，单位米；自由站不提供。
Baseline.id	id / String	观测记录唯一标识。
Baseline.from / to	from / to / String	有方向的基线端点；观测为终点减起点。
Baseline.vector	vector / Geometry.ENU	east、north、up，单位米。
Baseline.covariance	covariance / Geometry.Matrix3	必填，有限、对称、正定，单位平方米。

控制与连通性

核心请求固定 datum=external，每个独立连通分量都需固定控制点。没有控制约束的分量不能通过核心入口作为自由网解算。需要自由网或随机控制点时，应转到高级 JSON 协议。

相同站对可以有多条独立观测，但每条记录应使用不同 ID。复制同一观测增加权重会改变统计模型，不能作为增加独立冗余度的替代。

方向示例

```
// A=(0,0,0), B=(10,2,0.5)
// A -> B = (10,2,0.5); reverse the sign for B -> A.
var observation = new Geometry.ENU(10, 2, 0.5);
var covariance = Geometry.Matrix3.fromStdDev(.01, .01, .02);
```

五、输入输出模型（续）

2. GeodeticNetworkProblem

WGS84 输入顶层也包含 stations 和 baselines。控制坐标与基线坐标系原点分别设置：exactWGS84 固定站点位置，frameWGS84 决定一条基线向量的站心 ENU 方向。两者用途不同。

Java 字段	JSON 字段 / 类型	约束与说明
Station.id	id / String	站点唯一标识。
Station.exactWGS84	exact_wgs84 / WGS84Coordinate	提供即作为精确控制站；自由站只提供 ID。
Baseline.id / from / to	id / from / to / String	观测标识与有方向的站点引用。
Baseline.vectorENU	vector_enu / ENU	从起点到终点的向量；单位米。
Baseline.covarianceENU	covariance_enu / Matrix3	在本基线站心 ENU 轴上的协方差，单位平方米。
Baseline.frameWGS84	frame_wgs84 / WGS84Coordinate	核心入口每条基线必填，不从 exactWGS84 自动推导。

大地坐标字段

Java 字段	JSON 字段	单位与范围
latitudeDeg	latitude_deg	度，[-90,90]，北纬为正
longitudeDeg	longitude_deg	度，[-180,180]，东经为正
ellipsoidHeight	ellipsoid_height_m	米，WGS84 椭球高

WGS84Coordinate 构造顺序为纬度、经度、椭球高。不得把角度弧度、高程异常或海拔直接混用；每条基线的协方差必须与其向量处于相同坐标轴。

坐标系处理

原生库将各基线 ENU 观测及其协方差转换到地心坐标进行平差，再输出 WGS84/ECEF 站点结果。基线 adjustedENU、residualENU 和 standardized 仍位于该条输入基线的站心 ENU 坐标系。完整示例见附录 B。

五、输入输出模型（续）

3. Geometry 与 Matrix3

类型 / 方法	说明
Geometry.ENU	east、north、up；用于位置、向量、标准差时单位米，标准化残差无量纲。
Geometry.ECEFCoordinate	x、y、z；对应 x_m、y_m、z_m，单位米。
new Matrix3(double[9])	9 个行优先元素；构造与 setData 会复制数组，getData 返回副本。
Matrix3.at(row, column)	索引为 row×3+column，行列均为 0、1、2；越界抛 IndexOutOfBoundsException。
Matrix3.diagonal(vE,vN,vU)	参数直接作为方差；不进行正定性检查。
Matrix3.fromStdDev(sE,sN,sU)	将标准差平方为方差；Java 拒绝负数和非有限输入，但允许 0。
new Matrix3()	默认零矩阵，不满足核心协方差 required 策略。

```
var diagonal = Geometry.Matrix3.fromStdDev(.01, .015, .02);
var correlated = new Geometry.Matrix3(new double[] {
    .0001, .00002, .00001,
    .00002, .000225, -.000015,
    .00001, -.000015, .0004
});
```

fromStdDev 接受零不表示零协方差合法；原生仍要求正定。本轮零矩阵被 invalid_covariance 拒绝。非常大的有限标准差平方也可能溢出，后续 JSON 序列化会拒绝非有限值。

4. AdjustOptions

字段	缺省值	行为
timeoutMs / Long	null	0 也表示不限时；合法上限 9223372036854 ms，位于信封顶层 timeout_ms。
robust / boolean	false	true 转成 options.robust={}, 启用原生默认 Huber 参数。

核心请求固定使用 dense 求解、external 基准、required 协方差策略和 station-blocks 协方差输出。

station-blocks 提供站点精度与残差诊断，省去完整参数协方差矩阵的输出；并不把核心求解器变成稀疏算法。

五、输入输出模型（续）

5. 结果顶层与站点

ENUNetworkResult 与 GeodeticNetworkResult 顶层均只有 stations、baselines、diagnostics、warnings。协议中的其他字段被 Gson 忽略；需要完整矩阵及交叉协方差时必须保留原始 JSON。

模型 / 字段	JSON 字段	含义
ENU Station.id	id	与输入关联的站点 ID
ENU Station.position	position	平差坐标 ENU，米
ENU Station.stdDev	stddev	E/N/U 标准差，米；先检查可用性
WGS84 Station.id	id	与输入关联的站点 ID
positionWGS84	position_wgs84	纬度、经度、椭球高
positionECEF	position_ecef	地心 XYZ，米
stdDevXYZ	stddev_xyz	ECEF X/Y/Z 标准差，米；不是 ENU 标准差
mXYZ	m_xyz	$\sqrt{\sigma X^2 + \sigma Y^2 + \sigma Z^2}$ ，三维点位中误差，米
wgs84Absolute	wgs84_absolute	是否锚定到绝对 WGS84 基准

6. 基线结果

ENU 字段 / WGS84 字段	含义
id / from / to	保留观测标识和端点，便于追踪异常基线。
adjusted / adjustedENU	平差后的基线向量，米。
residual / residualENU	观测值减平差值，米；不要颠倒残差符号。
standardized	各方向标准化残差，无量纲；先检查 residualDiagnosticsAvailable。
weight	整条基线的鲁棒权值；普通平差为 1，低于 1 表示降权。

固定站精度零值源于“坐标被视为精确”的模型约束，不能解释为控制点实际测量完全无误差。输入协方差偏小或偏大均会影响整体随机模型检验。

六、质量诊断与错误处理

1. NetworkDiagnostics

Java 字段	JSON 字段	解释
sigma0	sigma0	后验单位权标准差；无正自由度时通常为 1。
degreesOfFreedom	degrees_of_freedom	核心模型为观测分量数减自由坐标参数数。
converged	converged	是否收敛；鲁棒迭代可正常返回但为 false。
stationCovarianceAvailable	station_covariance_available	为 true 才读取站点标准差、mXYZ 等精度。
residualDiagnosticsAvailable	residual_diagnostics_available	为 true 才解释 standardized。
globalTest	global_test	可为空；含 passed 和 confidence。

globalTest 是整体随机模型的双侧卡方检验摘要。无正自由度、发生鲁棒降权或估计方差分量时可能为空，表示未执行，不能当作通过。passed=false 可能由残差与给定协方差不相称导致，并不直接证明坐标求解错误。

2. 验收顺序

建议依次检查：对象与站点数量完整；坐标有限；converged=true；warnings 已评审；需要的精度/残差诊断可用；适用的整体检验通过；最后对照业务坐标真值、允许误差与时延要求。不要只根据“没有异常”验收。

本轮合成粗差样本使用默认 Huber，converged=true，粗差权值约 0.146，三轴偏差均约 0.002887 m。降权后 globalTest 为空，并返回省略整体检验的说明；这是预期行为。测试同时断言收敛、偏差小于 0.005 m、正常观测未被降权且无迭代上限告警。

六、质量诊断与错误处理（续）

3. AdjustException 与常见错误码

code 用于分支判断，field 和 id 可为空；getMessage() 返回异常说明。原始字符串入口需显式调用 JsonUtils.parseResponse 或自行检查 ok。JNA 加载/链接错误不保证包装为 AdjustException。

错误码	触发与处理	本轮覆盖
invalid_request	空输入、NUL、非法代理字符、非有限值或超时参数越界；修正请求。	已测
request_too_large	超过 64 MiB UTF-8；拆分业务批次或减少请求。	Java 传输边界已测
invalid_json	JSON 语法错误；由原生返回错误信封。	已测
invalid_response	原生响应信封类型、字段或尾部内容不合法。	人工响应已测
invalid_covariance	缺失、零矩阵或非对称协方差；应提供正定矩阵。	已测
duplicate_station	重复站点 ID；清理输入。	已测
unknown_station	端点不存在；核对 from/to。	已测
disconnected_network	无控制约束的分量；补齐控制点或使用自由网协议。	已测
invalid_problem	站网业务输入不合法；如缺失 WGS84 基线参考原点。	已测
rank_deficient / insufficient_redundancy	秩亏或冗余度不足；检查约束、网络 and 所请求诊断。	协议参考，未专项测
deadline_exceeded / cancelled	协作式截止或取消；不能保证立即中断稠密分解。	未实测运行超时
not_converged / internal_error / internal_panic	原生失败；记录输入与版本后排查。非收敛也可在成功结果中报告。	错误码未故障注入

字段级错误信息按原生值保留。缺失协方差实测为 field=baselines.covariance、id=AB1；所有基线 frameWGS84 缺失时，本轮返回 field=common_frame_origin_wgs84。错误字段可能随原生校验顺序变化，不宜依赖 message 文本。

七、JSON 协议与 JNA 绑定

1. 强类型请求信封

```
{
  "problem": { "stations": [], "baselines": [] },
  "options": {
    "solver": { "method": "dense" },
    "datum": "external",
    "covariance_policy": "required",
    "covariance": "station-blocks"
  },
  "timeout_ms": 5000
}
```

上例展示信封结构，空站网仅为字段示意。WGS84 将 problem 换为 geodetic_problem；robust=true 时在 options 中加入 robust={}。timeout_ms 省略或为 0 表示不限时。

2. 完整 JSON 接口

```
var request = java.util.Map.of(
    "problem", problem,
    "options", java.util.Map.of("covariance", "full"));
String raw = sdk.startAdjust(JsonUtils.toJson(request));
JsonUtils.Response envelope = JsonUtils.parseResponse(raw);
System.out.println(envelope.result().get("covariance"));
```

字符串入口不注入 dense/external/required/station-blocks 配置；未指定的选项遵循实际原生版本默认值。自由网、随机先验、稀疏求解及方差分量估计等需查阅配套原生协议，本轮仅专项对照了 full 协方差入口的核心输出一致性。

3. JsonUtils 工具接口

方法	作用
toRequestJson(problem, options)	ENU / WGS84 两个重载，构造核心请求。
toJson(Object)	序列化对象，拒绝 NaN、Infinity；不自动检查字节上限。
validateInput(String)	检查非空、UTF-8 字节上限、原始 NUL 和代理字符；语法交给原生。
parseResponse(String)	校验成功/失败信封；成功返回 Response(json,result)。
decode(String, Class<T>) / decode(Response, Class<T>)	成功 result 映射为目标类型；失败转换为 AdjustException。

七、JSON 协议与 JNA 绑定（续）

4. 响应语义

```
// Success
{"ok":true,"result":{"stations":[],"baselines":[]}}
// Failure
{"ok":false,"result":null,"error":{"
  "code":"invalid_covariance",
  "message":"...", "field":"baselines.covariance", "id":"AB1"
}}
```

parseResponse 要求根节点为对象、ok 为布尔值；成功时 result 为对象且 error 为空；失败时 error 必须含非空字符串 code/message，可选 field/id。它会拒绝尾随内容，但不是完整业务结果 schema 校验器：例如成功 result={} 可通过信封检查，调用方仍需检查必要结果字段。

5. 当前 Java 原生映射

```
public interface AdjustLibrary extends com.sun.jna.Library {
    String getVersion();
    String startAdjust(String requestJson);
    String startAdjustWgs84(String requestJson);
}
```

默认实例是 AdjustLibrary.INSTANCE。代码另有名为 INSTANTCES 的 synchronizedLibrary 包装实例，但默认服务并不使用它。直接使用原生接口将绕过 JsonUtils 的传输和结果校验。

6. C ABI 与返回内存

```
char *getVersion(void);
char *startAdjust(char *request_json);
char *startAdjustWgs84(char *request_json);
char *adjustValidate(char *request_json);
void adjustFree(char *value);
```

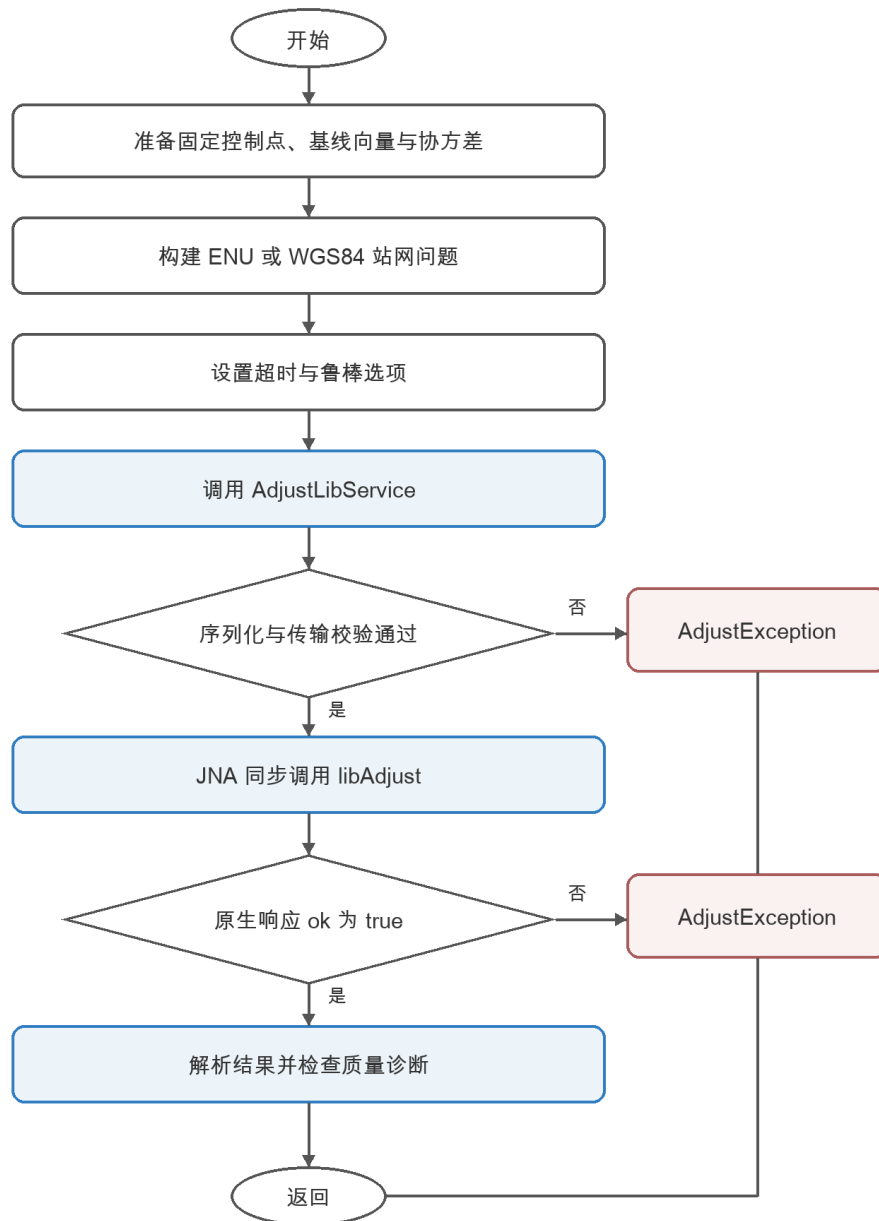
以上声明来自本机原生接口头文件；这五个符号已在 macOS 交付库核对。输入为借用的 UTF-8 NUL 结尾字符串，返回值由原生分配，约定使用 adjustFree 释放。Java 当前只绑定前三项，且直接返回 String，未保留待释放指针。

当前 Java 包不会释放每次原生返回的 C 字符串，getVersion 也包含在内。长期或高频运行前应完善 Pointer 接收与 finally 调用 adjustFree 的所有权封装；普通 Java GC 无法代替该释放协议。本轮未修改 SDK 源码。

八、调用流程

下图描述强类型入口。原始 JSON 入口到达原生后直接返回信封，由调用方处理 ok、result、error。

图中业务成功仍需经过收敛、告警与质量判定。



九、运行约束与故障排查

1. 并发、超时与资源

服务同步执行，调用期间不得修改输入列表、坐标或矩阵。并发任务应分别构造输入；本轮默认共享服务在 4 线程、80 次小网调用中与串行结果一致。该样本不等同于全部原生算法路径的线程安全或吞吐验收。

timeoutMs 由原生协作式检查，稠密矩阵分解不能被立即强制打断。Java Thread.interrupt 或 Future.cancel 不会自动取消原生工作。若业务必须限制硬超时，应考虑进程隔离；同时为线程池设置容量和排队上限。

核心固定采用稠密求解。站点规模、控制数量、连接关系和观测相关性都会改变代价；本轮只测 10/50/150 站的双观测星形网络。64 MiB 是请求传输上限，不是可安全求解的容量保证。

2. 常见问题

现象	核查与处理
UnsatisfiedLinkError	检查平台架构、资源目录、jna.library.path、动态依赖及文件访问权限；在新 JVM 中验证实际版本。
依赖解析失败	执行 ./mvnw install，核对 com.navfirst:adjust-java-lib:1.0.0 与仓库配置。
converged=false / warnings 非空	检查粗差、观测权重和迭代配置，不要只看返回对象。
标准差全为 0	先区分固定站、可用性标志及无残差算例；零值不等于物理精度零。
坐标方向或数量级错误	核对纬经顺序、度/弧度、米/毫米、基线方向及 ENU 坐标轴。
内存持续增长	当前 C 字符串释放未绑定；需修复所有权并执行长时 RSS 验证。
需要完整误差传播	使用 full JSON 返回协方差；各站 stdDev 不能恢复站间相关性。

附录 A：完整 ENU 调用示例

文件名 EnuExample.java。示例已从普通依赖 JAR 编译；运行输出的 B 坐标应接近 (10.4,2.4,0.7)

m。工程运行依赖 JNA 与 Gson。

```
import com.navfirst.adjust.lib.domains.*;
import com.navfirst.adjust.lib.domains.Geometry.*;
import com.navfirst.adjust.lib.services.AdjustLibService;
import com.navfirst.adjust.lib.services.impl.AdjustLibServiceImpl;
import java.util.List;

public class EnuExample {
    public static ENUNetworkProblem problem() {
        var a = ENUNetworkProblem.Station.builder().id("A")
            .fixed(true).knownENU(new ENU(0, 0, 0)).build();
        var b = ENUNetworkProblem.Station.builder().id("B").build();
        var ab1 = ENUNetworkProblem.Baseline.builder()
            .id("AB1").from("A").to("B")
            .vector(new ENU(10, 2, 0.5))
            .covariance(Matrix3.fromStdDev(1, 1, 1)).build();
        var ab2 = ENUNetworkProblem.Baseline.builder()
            .id("AB2").from("A").to("B")
            .vector(new ENU(12, 4, 1.5))
            .covariance(Matrix3.fromStdDev(2, 2, 2)).build();
        return ENUNetworkProblem.builder()
            .stations(List.of(a, b)).baselines(List.of(ab1, ab2)).build();
    }

    public static void main(String[] args) {
        AdjustLibService sdk = new AdjustLibServiceImpl();
        ENUNetworkResult result = sdk.startAdjust(problem());
        var d = result.getDiagnostics();
        if (!d.isConverged()) {
            throw new IllegalStateException("Adjustment did not converge");
        }
        result.getStations().forEach(s -> {
            System.out.println(s.getId() + ": " + s.getPosition());
            if (d.isStationCovarianceAvailable()) {
                System.out.println("stdDev=" + s.getStdDev());
            }
        });
        System.out.println("globalTest=" + d.getGlobalTest());
        System.out.println("warnings=" + result.getWarnings());
    }
}
```


附录 B：完整 WGS84 调用示例

文件名 Wgs84Example.java。每条基线显式提供 frameWGS84；输出标准差位于全局 ECEF XYZ 坐标轴。

```
import com.navfirst.adjust.lib.domains.*;
import com.navfirst.adjust.lib.domains.Geometry.*;
import com.navfirst.adjust.lib.services.impl.AdjustLibServiceImpl;
import java.util.List;

public class Wgs84Example {
    public static void main(String[] args) {
        var origin = new WGS84Coordinate(30.5, 114.3, 50);
        var a = GeodeticNetworkProblem.Station.builder()
            .id("A").exactWGS84(origin).build();
        var b = GeodeticNetworkProblem.Station.builder().id("B").build();
        var ab1 = GeodeticNetworkProblem.Baseline.builder()
            .id("AB1").from("A").to("B").frameWGS84(origin)
            .vectorENU(new ENU(10, 2, 0.5))
            .covarianceENU(Matrix3.fromStdDev(.01, .01, .02)).build();
        var ab2 = GeodeticNetworkProblem.Baseline.builder()
            .id("AB2").from("A").to("B").frameWGS84(origin)
            .vectorENU(new ENU(10.01, 2.01, 0.52))
            .covarianceENU(Matrix3.fromStdDev(.01, .01, .02)).build();
        var problem = GeodeticNetworkProblem.builder()
            .stations(List.of(a, b)).baselines(List.of(ab1, ab2)).build();
        var result = new AdjustLibServiceImpl().startAdjustWgs84(problem);
        var d = result.getDiagnostics();
        System.out.println("converged=" + d.isConverged());
        result.getStations().forEach(s -> {
            System.out.println(s.getId() + ": " + s.getPositionWGS84());
            System.out.println(s.getPositionECEF());
            if (d.isStationCovarianceAvailable()) {
                System.out.println("stdDevXYZ=" + s.getStdDevXYZ());
                System.out.println("mXYZ=" + s.getMXYZ());
            }
        });
        System.out.println("warnings=" + result.getWarnings());
    }
}
```

附录 C：集成检查表

阶段	检查内容
版本与分发	核对 pom 与依赖坐标；记录实际原生版本与二进制 SHA-256。
数据建模	固定控制点、站点 ID、基线方向、坐标轴、度与米、有效协方差。
强类型接口	仅依赖当前公开字段；options 可空；每条 WGS84 基线提供 frameWGS84。
结果验收	非空有限坐标、收敛、精度可用性、整体检验及业务真值精度。
高级接口	保留完整响应，按匹配原生版本解释矩阵、自由网和随机先验。
部署运行	动态库平台匹配、UTF-8、受控线程池、原生返回内存释放、超时边界。
测试交付	保留输入、独立答案、参数、结果、异常与环境。

集成注意事项

不要把随机控制先验直接改成 exactWGS84 或 fixed=true，否则会改变约束和结果精度。核心结果省略完整相关协方差；需要误差传播、自由网或分组估计的业务应保留原始 JSON 路径。

正式发版应同步修订 API 文档、版本号、二进制清单与测试报告。本指南记录当前源码和本机合成样本的实际行为；现场精度和长期运行应使用业务数据另行验证。